

SDM26 Cellular Telemetry Pipeline — End-to-End Proof Report

Helios · branch `feat/telemetry-pipeline` · 2026-06-12 · hosted project `dłmyixonuyckxkknolku`
(Helios Vault, East US Ohio) · client: Mac, residential WiFi, Mountain Time

VERDICT: The telemetry architecture works, proven end-to-end on production infrastructure. A 180-second sustained run drove synthetic SDM26 telemetry as binary HTTP/1 frames through the deployed edge function into staging, through the new Rust compactor into zstd parquet in storage, and out to a live Realtime subscriber — with **zero transport errors, zero lost frames, zero duplicate rows, and bit-exact integrity on all 44,594 verified samples across all 38 channels**. A second run under deliberate cellular chaos (20% packet drop, 5% duplication, 300 ms jitter) recovered **every** window through the retry queue — again bit-exact in parquet. The database returned to idle baseline after each run. One latency target (live-view p95) was missed from this test location; the fix is implemented and the remaining work is itemized.

1. What was proven (and what wasn't yet)

Pipeline stage	Mechanism	Status
Wire protocol	HTTP/1 binary frames (36 B header + 1 s windows; f32 / i16fp encodings)	PROVEN
Device auth	HMAC-SHA256 over raw body (<code>x-http-device</code> / <code>x-http-signature</code>)	PROVEN
Ingest	Edge function: decode → Arrow IPC → idempotent staging upsert → seq acks	PROVEN
Retry / dedup	PK (<code>session</code> , <code>group</code> , <code>seq</code>); dups reported in ack, never re-staged	PROVEN (69/69 injected dups)
Live view	Realtime Broadcast <code>telemetry:live:{session}</code> , 1 Hz downsampled frame	PROVEN (552/552 delivered)
Compaction	<code>helios-compactor</code> (new Rust crate): staging → zstd parquet → storage, crash-safe	PROVEN (8.1–8.7× compression)
1 Hz downsample	Mean-per-second chunks → <code>telemetry.downsampled_1hz</code> (web read path)	PROVEN (10 chunks)
Storage integrity	Full differ: every sent sample vs parquet, at wire	PROVEN (100% bit-

	resolution	exact)
Web decodability	Staged Arrow IPC decoded with apache-arrow JS (Helios Lite path)	PROVEN
Retention	pg_cron telemetry-prune-staging every 5 min (pg_cron live on hosted)	SCHEDULED
Vault isolation	Separate schema/bucket, RLS, zero Vault references; DB returned to baseline	VERIFIED
Chaos resilience (drop/dup/jitter)	S3: 20% POST drop + 5% dup + 300 ms jitter, seq/ack retry queue	PROVEN (0 loss, integrity exact)
Cellular bandwidth profiles	cat_1/cat_4 throttles — needs generator	NOT YET (planned §5.2)
Vault canary under load	Scenario S5 — needs bench app	NOT YET (planned §5.3)
Real ESP32 hardware	Firmware implements HTP/1 per docs/telemetry-wire-protocol.md (C sketch included)	NOT YET (hardware)

2. The proof run (180 s sustained, hosted production path)

Session ed46dea0...: 3 concurrent group streams (22 ch @ 10 Hz i16fp/f32, 5 ch GPS @ 10 Hz f32, 11 ch thermal @ 1 Hz i16fp) = 38 channels, 1 window/s/group, every 10th frame deliberately retransmitted, live subscriber attached, compactor running with 60 s min-span.

2.1 Ingest

Metric	Result	Note
Frames offered / acked	501 / 501	zero transport errors over 180 s
Deliberate duplicate retries	51 / 51 correct	all identified as dup, none re-staged
Ack RTT p50 / p95 / p99	450 / 669 / 907 ms	home WiFi → Ohio; well under the 1 s send cadence
Wire bytes (180 s, 38 ch)	132,776 B (~0.72 KB/s)	i16fp packing; trivially fits cat_1 (2 Mbps)

Staging integrity	exact	contiguous seqs per group, no gaps, no dups
-------------------	-------	---

2.2 Compaction (new `crates/helios-compactor`)

Wave	Chunks	Rows	Staged bytes	Parquet bytes	Compression
≈ 65 s	3	178	449,120	51,565	8.7×
≈ 125 s	3	168	427,296	50,012	8.5×
≈ 185 s	3	154	385,152	47,312	8.1×
session-end flush	1	1	4,608	9,533	0.5× (footer-dominated tail)

Staging fully drained (0 pending rows) after session end; 10 parquet objects + 10 downsampled-1 Hz chunks written. Crash-safety is structural: deterministic object keys from the seq range, upsert uploads, object size verified by read-back *before* rows are marked compacted, and marking targets the exact seq list (never a blind range, so a late retry inside the range can never be marked without being in parquet).

2.3 Live broadcast (Realtime subscriber, `supabase-js`)

Metric	Result	Note
Frames received	552 / 552	= 501 originals + 51 dup retransmits; zero loss
e2e latency p50 / p95 / p99	488 / 905 / 1245 ms	generator timestamp → subscriber receipt, same host clock

Finding & fix: p95 905 ms missed the S1 target (<500 ms) because the function broadcast only *after* the staging insert completed — so live-view latency carried the full insert RTT (ack p50 was 450 ms to Ohio). **Fixed** (commit below): the broadcast now fires concurrently with the insert and is awaited just before the response, so it no longer waits behind the DB write — expected to cut broadcast latency by roughly one insert-RTT. The reorder is committed and verified functionally correct on the local stack (ingest acks, dedup intact, broadcast fires); the hosted redeploy + re-measurement is pending a fresh CLI token (the working session's token expired). Not a viability concern regardless: drivers watch trends and 0.5–1 s telemetry-to-screen is competitive.

2.4 Integrity (the core claim)

The new `helios-compactor verify` subcommand downloaded all 10 parquet objects and diffed every transmitted sample (from the generator's JSONL dump) against the staged value on its exact time-grid slot:

Check	Result
-------	--------

Samples compared (all 38 channels, all 501 windows)	44,594
Bit-exact at wire resolution (f32 bit-exact; i16fp exact post-quantization)	44,594 (100%)
Within-tolerance (non-exact)	0
Mismatched / missing	0

This closes the loop: what the car sends is exactly what analysis reads, through wire encode → edge decode → Arrow IPC → parquet → download → decode.

2.5 Chaos run — cellular resilience (scenario S3)

A second 120 s run injected a simulated bad cellular link: each 1 s window passed through a chaos wrapper applying a 20% probability of dropping the POST entirely, 5% of duplicating it (modem retransmit), and 0–300 ms of random added latency. A bounded 32-window seq/ack retry queue held un-acked windows and re-sent them.

Metric	Result	Note
POSTs dropped by the chaos link	76	~20% of deliveries, as configured
POSTs duplicated	11	all 11 correctly deduped server-side
Retry-queue overflow drops	0	queue never saturated at this rate
Windows un-acked at end	0 / 0 / 0	retry queue drained completely — zero data loss
Compacted-parquet integrity	33,317 / 33,317 exact	differ vs dump: 0 mismatch despite 76 drops
Ack RTT p50 / p95	538 / 804 ms	jitter-inflated but stable

This is the decisive cellular result: under one-in-five packet loss the protocol's nack/retry design delivered *every* sample, and the compactor wrote them to parquet bit-exact. Compression actually improved on the sparser fast-group chunk (up to 13×).

2.6 Cleanliness

After each run, bulk-deleting the session's storage objects and cascade-deleting the session left **0** staging rows, **0** downsampled rows, and **0** storage objects. Hosted telemetry tables confirmed back to idle (0 sessions/staging/downsampled; only the 92 seed registry rows + 1 channel set remain).

Database measured 61 MB vs 59 MB baseline immediately after the first cleanup — the 2 MB delta is dead tuples awaiting autovacuum, not retained data (live telemetry row count: 0).

3. Effects on the Supabase project

Everything below lives in one shared Supabase project (`d1myixonuyckxkknolku`, the same project as the Helios Vault — that co-location is deliberate, so impact is measurable). The changes split cleanly into **persistent** (added by deploy, still present, removed only by teardown) and **transient** (created during tests, fully cleaned up).

3.1 Persistent additions (present now, until teardown)

Surface	What was added	Footprint
Database schema	New schema <code>telemetry</code> : 9 tables, 11 indexes, 1 function (<code>prune_staging</code>)	~424 kB at idle
Seed data	<code>channel_registry</code> (92 rows) + <code>channel_sets</code> (1 row) — the only non-empty telemetry tables at rest	part of the 424 kB
Row-level security	RLS enabled on all 9 tables; 9 read policies (authenticated read, service-role write, anon nothing)	—
Storage	1 private bucket <code>telemetry-sessions</code> + 1 scoped read policy on <code>storage.objects</code>	0 objects at idle
Scheduled job	<code>pg_cron</code> job <code>telemetry-prune-staging</code> (<code>*/5 * * * *</code> , prunes compacted rows past 1 h)	1 cron row
PostgREST	<code>telemetry</code> added to exposed schemas (<code>public, ..., pdm, pm, games</code> → <code>+telemetry</code>)	config only
Edge function	<code>telemetry-ingest</code> deployed + 1 secret <code>TELEMETRY_HMAC_KEY</code>	1 function
Migration history	2 rows recorded in <code>supabase_migrations.schema_migrations</code> (applied via management API, history is drifted)	2 rows

Net persistent database growth from deploy: **~0.4 MB** (schema scaffolding + seed). Nothing else on the project — no existing table, policy, bucket, cron job, or function — was altered.

3.2 Transient load during the two test runs (now fully removed)

Resource	Peak / total during tests	State now
Staging rows (<code>staging_chunks</code>)	~501 peak (proof), ~353 (chaos)	0 (cascade-deleted)

Parquet objects in storage	16 total (10 + 6), ~0.5 MB	0 (bulk-deleted)
Downsampled chunks	16 total (10 + 6)	0
Realtime Broadcast messages	~1,100 (552 proof + chaos)	not retained (ephemeral)
Edge function invocations	~950 (501 + ~280 deliveries + dups + verify)	billed events only
Storage egress (verify downloads)	~1 MB (16 parquet reads)	—
Postgres connections	compactor 1, bench REST short-lived, 1 realtime sub	0

3.3 Database size, before / during / after

Moment	pg_database_size	Telemetry live rows
Pre-deploy baseline	59 MB	n/a (schema absent)
After deploy, at idle	~59 MB (+0.4 MB schema)	93 (seed only)
During 180 s proof run	~60–61 MB	up to ~500 staging
After cleanup	61 MB	0 working rows

The residual 61 vs 59 MB is dead-tuple bloat from the insert/delete churn, which autovacuum reclaims; it is not retained telemetry. Steady-state cost of a season is bounded by the §6 projection (tens of MB), and the 1 h staging retention + compaction keep the hot set small.

3.4 Effect on the Vault (the reason for co-location)

The Vault's `pdm / pm / games` schemas, their tables, policies, storage bucket, and data were **not touched**: telemetry writes are confined to schema `telemetry`, bucket `telemetry-sessions`, channels `telemetry:*`, and function `telemetry-ingest`; there are zero foreign keys or shared policies. Vault row counts were identical before and after (`pdm.versions` 13,180; `pdm.files` 13,147; `pm.tasks` 244). The formal *quantitative* canary — measuring Vault query p95 *while* telemetry is under load (scenario S5) — still needs the bench app and is the one isolation claim not yet measured under concurrency; structural isolation, however, is in place and verified.

4. Defects found and fixed along the way

#	Defect	Impact	Fix (commit)
1	Schema migration granted <code>service_role</code> no table privileges (RLS bypass $\neq$ table grants)	Every service-role read/write failed with "permission denied" — pipeline dead on arrival	Explicit grants + default privileges (29f6e67)
2	Teardown deleted storage rows via SQL; current Storage installs a <code>protect_delete()</code> trigger that rejects this	Teardown silently left the bucket; "clean removal" claim false	Storage-API empty+delete documented; script aborts upfront if bucket exists (a28251e)
3	Edge decoder expected per-channel <code>rate_hz</code> ; the seeded contract puts it at group level	Every seeded channel set rejected with HTTP 500	Group-level rate canonical, per-channel must agree; tests 12/12 (e803d1a)
4	Compactor's RFC3339 cutoff (+00:00) decoded as a space in the PostgREST URL	Every staging scan failed HTTP 400	Z-suffix timestamp format (2327dc6)

Also noteworthy, not defects: the platform injects the *legacy JWT* service-role key into edge functions (bearer comparisons must use it, not the `sb_secret_...` form); node 20 lacks global WebSocket (realtime needs an explicit ws transport); migration history is drifted, so hosted DDL goes through the management API with manual history rows, never `db push`.

5. Operational state & safety

- **Backups:** full `pg_dump` set (roles/schema/20 MB data) in `~/helios-backups/pre-telemetry-2026-06-12/`, restore-verified into a platform-initialized scratch DB (row counts match hosted exactly: `pdm.versions` 13,180; `pdm.files` 13,147; `pm.tasks` 244). PM-schema JSON dump additionally in `~/helios-backups/pm-2026-06-12/` (22 tables, 1,468 rows).
- **Vault isolation:** all telemetry writes confined to schema `telemetry` + bucket `telemetry-sessions`; RLS on every table; anon has zero grants; reads-only exposure for `authenticated`.
- **Retention:** `pg_cron` prune job live (5-min cadence, 1 h retention of compacted rows); compactor has a `--prune` fallback flag.
- **Deployed:** both migrations (recorded in history), seed (92 channels, set 1), edge function with `TELEMETRY_HMAC_KEY` secret, `telemetry` in PostgREST exposed schemas.

6. Quota projection (rough, measured basis)

Quantity	Measured basis	30-min session	Season (40 sessions ≈ 20 h)
Wire (cellular up)	0.72 KB/s	~1.3 MB	~52 MB

Parquet at rest	~0.83 KB/s	~1.5 MB	~60 MB
Staging (transient)	~2.5 KB/s, pruned at 1 h bounded < 10 MB at any moment		
Broadcast messages	~3/s	~5,400	~216,000

All far below Pro-plan quotas (storage 100 GB, realtime messages in the millions/month). The formal "<50% of every quota" check (handoff §5.4) runs in the bench app with a config-file quota table.

7. What remains before the car runs (handoff §5)

- **crates/helios-telemetry-gen** — the reference HTP/1 client the ESP32 firmware copies: CSV replay (real MoTeC logs), chaos wrapper (drop/jitter/reorder/dup), cat_1/cat_4 bandwidth throttles, bounded retry queue. The Python bench driver proved the protocol; this makes it product-grade and runs S2–S4.
- **apps/telemetry-bench** — control panel, live tiles + strip chart, metrics dashboard, DB-impact panel with the **Vault canary** (S5: pdm read p95 under telemetry load within 10% of baseline).
- **Broadcast latency re-measurement** — the publish-before-insert fix is committed and locally verified; deploy it to hosted (needs a fresh CLI token) and confirm the p95 drop.
- **Bandwidth throttles + Vault canary (S5)** — cat_1/cat_4 profiles and the concurrent Vault-read p95 measurement, both in the bench app.
- **ESP32 firmware** — implement HTP/1 per the protocol doc's C sketch; HMAC key provisioning.
- **Tail-chunk economics** — 1-row flush chunks cost more in parquet footer than data (9.5 KB for 1 row); consider merging tails into the prior chunk or a min-rows floor.

Appendix A — Artifacts

- Branch `feat/telemetry-pipeline` — infra, edge function, compactor, and bench tooling all pushed.
- New crate: `crates/helios-compactor` (run + verify subcommands, 4 unit tests).
- Tools: `infra/telemetry-supabase/scripts/hosted-smoke-bench.py` (binary HTP/1 generator + chaos wrapper), `scripts/live-subscriber.mjs` (broadcast latency probe), `scripts/teardown.sql` (hardened).
- Raw run reports: `bench-results/hosted-smoke-20260612-18*.json`; example committed at `docs/telemetry-hosted-smoke-example.json`.
- Canonical state doc: `docs/telemetry-bench-HANDOFF.md` (updated in place).

Generated 2026-06-12 by the Claude Code session that performed the work. All measurements above were taken against the live hosted project during this session; nothing is estimated except the explicitly-labeled season projections.